

Σχεδίαση Και Ανάπτυξη Πληροφοριακών Συστημάτων Συνολική Αναφορά

Ελισσάβετ Πολύζου, Δήμητρα Ραπτοπούλου, Χρυσούλα Νικολοπούλου,
Ερμής Κάτσης, Δημήτρης Ιτιμούδης



**TECHNICAL
UNIVERSITY
OF CRETE**

4/1/2026

Μάθημα: Σχεδίαση Και Ανάπτυξη Πληροφοριακών Συστημάτων
(ΠΛΗ 302)

ΑΜ Ομάδας: 2022030131, 2022030132,
2022030156, 2022030171, 2022030145

Σχολή: ΗΜΜΥ/ Πολυτεχνείο Κρήτης

1. Περίληψη

Το παρακάτω κείμενο αποτελεί μία αναφορά σχετικά με την ανάλυση ενός συστήματος ηλεκτρονικής τραπεζικής υπηρεσίας E-Banking για την τράπεζα Bank of TUC. Στόχο μας αποτελεί η ανάλυση, σχεδίαση και, τέλος, υλοποίηση ενός πλήρους λειτουργικού πληροφοριακού συστήματος που θα ικανοποιεί τις καθημερινές ανάγκες του πελατειακού κοινού της τράπεζας (ιδιώτες και επιχειρήσεις), θα είναι ασφαλές για την αποφυγή υποκλοπών (identity theft κτλ) και ο σχεδιασμός του θα διευκολύνει μελλοντικές προσθήκες νέων μηχανισμών όπως SEPA Instant .

Για τον καθορισμό των απαιτήσεων και αναγκών του συστήματος αυτού, πήραμε συνέντευξη από την Μαρία Κοντογιάννη-Product Owner, Digital Channels (Bank of TUC Executive) η οποία ως τραπεζικός stakeholder μας διευκρίνισε τι αποκομίζει από το υπό ανάπτυξη σύστημα.

2. Οφέλη

Με την ανάπτυξη ενός Web Banking ακολουθούν πολλά οφέλη, όπως εξηγεί και η ίδια.

- ο Πρώτον, δεν επιβαρύνονται τα φυσικά καταστήματα από ουρές πελατών για συναλλαγές οι οποίες μπορούν, πλέον, να γίνουν μέσω μίας πλατφόρμας/application και συνεπάγεται από αυτό ένα καλύτερο ποσοστό ικανοποίησης πελατών.
- ο Μειώνεται το κόστος λειτουργίας καθώς εκτελούνται λιγότερες χειροκίνητες επεμβάσεις υπαλλήλων για καθημερινές συναλλαγές όπως πληρωμές λογαριασμών, καταθέσεις, ενημερώσεις υπολοίπων κλπ.
- ο Γίνεται άμεση καταγραφή κινήσεων και ανάλυση για εποπτικούς και κανονιστικούς λόγους.
- ο Αύξηση ικανοποίησης και πιστότητας μέσω εύχρηστου περιβάλλοντος και αυτοματισμών.(πάγιες εντολές, ειδοποιήσεις, statements κ.λπ.).
- ο Ευκολία επέκτασης μέσω σχεδιασμού που υποστηρίζει νέα πρωτόκολλα και προϊόντα.
- ο Βελτίωση KPIs (Key Performance Indicators) με μείωση κόστους εξυπηρέτησης (~30%), αύξηση ψηφιακών συναλλαγών (>70%) και περιορισμό λαθών και καθυστερήσεων – με θετικό αντίκτυπο στην αξιολόγησή της Executive και στις επιδόσεις του Digital Division.
- ο Βελτιωμένη διακυβέρνηση και διαφάνεια μέσω audit logs, statements και πλήρους ιχνηλασιμότητας– με ακριβές reporting και analytics για τη συμπεριφορά χρηστών.

3. Κίνδυνοι⇒Μέτρα Πρόληψης

Σε συνέχεια της συνέντευξης πληροφορηθήκαμε από την Product Owner τα κύρια ρίσκα που υπάρχει πιθανότητα να επέλθουν από ένα αντίστοιχο έργο και πιθανά μέτρα πρόληψης (που θα επεξηγούνται μετά το ⇒) του κάθε κινδύνου σύμφωνα με την εμπειρία της.

- ο Ως νέα ανάπτυξη, μπορεί να εμφανιστούν σφάλματα στη λογική συναλλαγών (balances, statements κ.λπ.) και να υπάρξει απώλεια εμπιστοσύνης χρηστών λόγω κακής εμπειρίας.⇒ Δημιουργία test datasets με συγκεκριμένα σενάρια (success, failure, edge cases) και καθημερινό testing με “Time Simulation” ώστε να εντοπίζονται έγκαιρα τα λάθη.
- ο Η ασυμβατότητα με μελλοντικά πρωτόκολλα, αν δεν προβλεφθεί σωστά, μπορεί να αυξήσει σημαντικά το κόστος συντήρησης.⇒ Χρήση Bridge pattern όπως ορίζει η πολιτική Bank of TUC, με καθαρό διαχωρισμό business logic και transfer mechanism (π.χ. IntraBank, SEPA, SWIFT).
- ο Η ελλιπής προσομοίωση χρόνου ενδέχεται να καταστήσει αναξιόπιστους τους ελέγχους αυτόματων λειτουργιών, μειώνοντας την αξιοπιστία των δεδομένων. ⇒ Προγραμματισμένα unit tests για αυτόματες λειτουργίες (τόκοι, πάγιες) και επαλήθευση ημερήσιας ροής πριν από κάθε release.
- ο Η μη ευθυγράμμιση με τις ανάγκες των χρηστών μπορεί να οδηγήσει σε ανασχεδιασμό και καθυστερήσεις, αν οι φοιτητές-αναλυτές δεν κατανοήσουν σωστά το banking domain. ⇒ Τακτικά validation meetings με stakeholder.
- ο Η υπερβολική τεχνική πολυπλοκότητα πέρα από το MVP scope μπορεί να προκαλέσει απώλεια ελέγχου χρόνου και ποιότητας.⇒ Ξεκάθαρος προσδιορισμός του scope

4. Κύριοι ρόλοι χρηστών του συγκεκριμένου συστήματος

4.1 Πελάτης

1. Φυσικό Πρόσωπο(Individual Customer): Ιδιώτης πελάτης που διατηρεί έναν ή περισσότερους προσωπικούς λογαριασμούς.
 - ο Κύριες δυνατότητες:
 - ο Επισκόπηση υπολοίπων και κινήσεων (statements).
 - ο Κατάθεση / Ανάλυση (Deposit / Withdrawal).
 - ο Μεταφορά χρημάτων σε άλλους λογαριασμούς της τράπεζας (Intra-bank transfer).
 - ο Πληρωμή λογαριασμών προς εταιρείες (Payment by RF code).
 - ο Δημιουργία πάγιων εντολών (Standing Orders) για περιοδικές πληρωμές ή μεταφορές.
 - ο Λήψη ειδοποιήσεων (π.χ. αποτυχία πάγιας, επιτυχής πληρωμή).

2. Επιχείρηση (Company Customer): Νομικό πρόσωπο που έχει έναν και μόνο έναν επιχειρηματικό λογαριασμό.

ο Κύριες δυνατότητες:

- ο Έκδοση λογαριασμών πληρωμής (Bills) με RF code προς πελάτες.
- ο Παρακολούθηση εξοφλήσεων.
- ο Προβολή κινήσεων του επιχειρηματικού λογαριασμού.
- ο Καταβολή τελών συντήρησης (χρεώνεται αυτόματα στο τέλος του μήνα).

Διαχειριστής (Admin):

Εσωτερικός υπάλληλος της τράπεζας, χωρίς προσωπικό λογαριασμό που επιβλέπει και διαχειρίζεται όλα τα δεδομένα του συστήματος.

ο Δυνατότητες:

- ο Διαχείριση πελατών (προβολή στοιχείων, αναζήτηση).
- ο Προβολή όλων των τραπεζικών λογαριασμών.
- ο Επισκόπηση και επαλήθευση συναλλαγών (Transactions & Statements).
- ο Διαχείριση πάγιων εντολών (ενεργοποίηση/απενεργοποίηση).
- ο Διαχείριση λογαριασμών πληρωμής εταιρειών.
- ο Εκτέλεση “Simulate Time Passing” – δηλαδή προσομοίωση ημερών για αυτόματες λειτουργίες (τόκοι, πάγιες, τέλη).

Η Τράπεζα (Bank System)

Αν και δεν είναι “χρήστης” με login, το σύστημα της τράπεζας εκτελεί αυτόματα πράξεις εκ μέρους των πελατών όπως:

- ο Υπολογίζει τόκους και αποδίδει τακτικά.
- ο Χρεώνει τέλη συντήρησης επιχειρηματικών λογαριασμών.
- ο Εκτελεί πάγιες εντολές στις καθορισμένες ημερομηνίες.
- ο Τηρεί audit logs για κάθε αυτόματη ενέργεια

5. Βασικές οντότητες (entities)

Έπειτα έγινε επεξήγηση από επιχειρηματική σκοπιά πως θα περίμεναν στην Bank of TUC τις βασικές οντότητες

5.1 Τραπεζικός Λογαριασμός (Bank Account):

• Τύποι λογαριασμών:

- Προσωπικός Λογαριασμός (Personal Account) → ανήκει σε φυσικό πρόσωπο, μπορεί να έχει συνδικαιούχους.
- Επιχειρηματικός Λογαριασμός (Business Account) → ανήκει σε μία εταιρεία, μόνο ένας ανά επιχείρηση.
 - Μοναδικό Αναγνωριστικό: IBAN (20 χαρακτήρες).
 - Κύρια πεδία:

- Υπόλοιπο (balance)
- Επιτόκιο (interest rate)
- Κατάσταση (ενεργός / ανενεργός)
- Κάτοχος (individual ή company)
- Ημερομηνία δημιουργίας
- *Επιχειρηματικοί κανόνες:*
 - Το υπόλοιπο αλλάζει μόνο μέσω συναλλαγής (Transaction).
 - Οι εταιρικοί λογαριασμοί χρεώνονται με τέλος συντήρησης (maintenance fee) κάθε μήνα.
 - Οι τόκοι υπολογίζονται καθημερινά και αποδίδονται στο τέλος του μήνα.

5.2 Συναλλαγή (Transaction)

Η πράξη που μεταβάλλει τα υπόλοιπα λογαριασμών.

- Τύποι:
 - Κατάθεση (Deposit)
 - Ανάληψη (Withdrawal)
 - Μεταφορά (Transfer)
 - Πληρωμή (Payment - μέσω RF code)
 - Αυτόματες (Interest / Fee transactions)
- Κύρια πεδία:
 - Κωδικός συναλλαγής (Transaction ID)
 - Εκτελεστής (Transactor: Customer / Admin / Bank)
 - Εμπλεκόμενοι λογαριασμοί (From / To IBAN)
 - Ποσό
 - Αιτιολογία (reason / description)
 - Χρονοσήμανση (timestamp)

 **Business rule:** κάθε συναλλαγή παράγει **τουλάχιστον ένα Statement** ανά εμπλεκόμενο λογαριασμό.

5.3 Κίνηση Λογαριασμού (Account Statement)

Το αποτύπωμα κάθε συναλλαγής.

- Πεδία
 - Ημερομηνία και ώρα
 - Είδος (Χρέωση ή Πίστωση)
 - Ποσό
 - Υπόλοιπο μετά τη συναλλαγή
 - Αιτιολογία
 - Transaction ID (σύνδεση με τη συναλλαγή)

 **Business rule:** οι κινήσεις τηρούνται **αντίστροφα χρονολογικά** (τελευταία πρώτα).

5.4 Λογαριασμός Πληρωμής (Bill)

Αντιπροσωπεύει έναν λογαριασμό που έχει εκδώσει μια εταιρεία.

- Πεδία:
 - Κωδικός πληρωμής RF (Reference Code)
 - Κωδικός λογαριασμού (Bill ID)
 - Εκδότης (Company)
 - Πελάτης (Customer)
 - Ποσό
 - Ημερομηνίες έκδοσης / λήξης
 - Κατάσταση (ενεργός / εξοφλημένος)

Business rules:

- μόνο ένας ενεργός κάθε φορά.
- Οι πληρωμές bill πραγματοποιούνται μόνο μέσω λογαριασμού πελάτη.

5.5 Πάγια Εντολή (Standing Order)

Εντολή αυτόματης εκτέλεσης συναλλαγών.

Τύποι:

- **Payment Order** → πληρωμή λογαριασμών με RF code.
- **Transfer Order** → περιοδική μεταφορά χρημάτων.

Πεδία:

- Κωδικός εντολής
- Τίτλος / Περιγραφή
- Ενεργό διάστημα (start / end)
- Συχνότητα (π.χ. κάθε μήνα)
- Μέγιστο ποσό (για Payment Order)
- Κατάσταση (ενεργή / ανενεργή / ληγμένη)

Business rules:

- Εκτελεστής θεωρείται το **σύστημα (Bank)**.
- Αν μια απόπειρα αποτύχει **3 φορές**, η εντολή παραμένει ενεργή, αλλά η τρέχουσα απόπειρα ακυρώνεται.

5.6 Τόκος (Interest)

Αυτόματη πίστωση που υπολογίζεται ανά λογαριασμό.

- Υπολογίζεται **καθημερινά** αλλά αποδίδεται **στο τέλος κάθε μήνα**.
- Καταχωρείται ως **συναλλαγή τύπου "Interest Credit"**.
- Ο τόκος υπολογίζεται βάσει του ημερήσιου υπολοίπου × επιτόκιο / 365.

5.7 Τέλη Συντήρησης (Maintenance Fees)

Αυτόματη χρέωση στους εταιρικούς λογαριασμούς

- Εφαρμόζεται μια φορά τον μήνα.
- Το ποσό κατατίθεται αυτόματα στον λογαριασμό της Τράπεζας.
- Εμφανίζεται ως συναλλαγή τύπου “Fee Debit”.

6. Λειτουργικές απαιτήσεις Συστήματος (Functional Requirements Systems)

Σε αυτή την ενότητα δίνονται οι λειτουργικές απαιτήσεις που καθορίζουν τις βασικές ενέργειες και συμπεριφορές που πρέπει να υποστηρίζει το σύστημα e-Banking της Bank of TUC. Κατηγοριοποιούνται ανά υποσύστημα και έχουν ιεραρχηθεί με βάση τη μεθοδολογία MoSCoW (Must, Should, Could, Won't), ώστε να αποτυπώνουν τις προτεραιότητες ανάπτυξης

6.1 Διαχείριση χρηστών (User Management)

Προτεραιότητα: MUST HAVE

- Είσοδος (login) με όνομα χρήστη και κωδικό.
- Διάκριση ρόλων: Individual, Company, Admin.
- Διαφορετικές λειτουργίες και μενού ανά ρόλο.

Προτεραιότητα: SHOULD HAVE

- Ο Admin μπορεί να δει/διαχειριστεί στοιχεία πελατών

6.2 Διαχείριση λογαριασμών (Accounts)

Προτεραιότητα: MUST HAVE

- Δημιουργία τραπεζικών λογαριασμών με μοναδικό IBAN.
- Ανάθεση λογαριασμού σε κάτοχο (Individual ή Company).
- Εμφάνιση υπολοίπου, επιτοκίου και κατάστασης (ενεργός/ανενεργός).
- Μηνιαία αυτόματη χρέωση τέλους συντήρησης για εταιρικούς λογαριασμούς.
- Αυτόματος υπολογισμός και απόδοση τόκων στο τέλος κάθε μήνα.

Προτεραιότητα: SHOULD HAVE

- Διαχείριση συνδικαιούχων (optional).
- Ο Admin μπορεί να απενεργοποιεί ή να ενεργοποιεί λογαριασμούς.

6.3 Συναλλαγές (Transactions)

Προτεραιότητα: MUST HAVE

- Κατάθεση (Deposit) χρημάτων σε λογαριασμό.
- Ανάληψη (Withdrawal) εφόσον υπάρχει επαρκές υπόλοιπο.
- Μεταφορά (Transfer) μεταξύ λογαριασμών της ίδιας τράπεζας.
- Πληρωμή (Payment) λογαριασμών επιχειρήσεων μέσω κωδικού RF.
- Κάθε συναλλαγή παράγει εγγραφή (statement).
- Αυτόματες συναλλαγές:
 - *Interest Credit* (πίστωση τόκου)
 - *Fee Debit* (χρέωση τέλους)

Προτεραιότητα: SHOULD HAVE

- Έλεγχος εγκυρότητας ποσών και υπολοίπου πριν από κάθε συναλλαγή, με μήνυμα λάθους σε περίπτωση ανεπαρκούς υπολοίπου ή μη έγκυρης τιμής ποσού.

6.4 Κινήσεις λογαριασμού (Account Statements)

Προτεραιότητα: MUST HAVE

- Δημιουργία **statement** για κάθε συναλλαγή και κάθε εμπλεκόμενο λογαριασμό.
- Εμφάνιση των εξής πεδίων: ημερομηνία, ποσό, τύπος (χρέωση/πίστωση), υπόλοιπο μετά τη συναλλαγή, αιτιολογία.
- Τα statements τηρούνται **αντίστροφα χρονολογικά**.

Προτεραιότητα: SHOULD HAVE

- Δυνατότητα φιλτραρίσματος ή εξαγωγής κινήσεων ανά περίοδο.
- Ο Admin έχει πρόσβαση σε statements όλων των λογαριασμών.

6.5 Λογαριασμοί πληρωμών (Bills)

Προτεραιότητα: MUST HAVE

- Οι επιχειρήσεις (Company Users) εκδίδουν λογαριασμούς πληρωμής (Bills) προς πελάτες με μοναδικό **RF code**.
- Κάθε Bill περιλαμβάνει: ποσό, ημερομηνία έκδοσης, ημερομηνία λήξης, κατάσταση (ενεργός / εξοφλημένος).
- Οι **πελάτες** πληρώνουν ενεργούς λογαριασμούς μέσω της λειτουργίας **Payment**.

Προτεραιότητα: SHOULD HAVE

- Έλεγχος από το σύστημα για :
 - ? Ληγμένο ή ανενεργό RF code
 - ? Διπλή πληρωμή (ίδιο RF σε κατάσταση "paid").

Προτεραιότητα: COULD HAVE

- Ο Admin μπορεί να φορτώνει μαζικά bills, να ελέγχει εξοφλημένα ή ληγμένα, και να επανενεργοποιεί RF codes εάν απαιτείται.

6.6 Πάγιες Εντολές (Standing Orders)

Οι πάγιες εντολές αντιπροσωπεύουν αυτοματοποιημένες ενέργειες πληρωμών ή μεταφορών που εκτελούνται σε τακτά χρονικά διαστήματα από το σύστημα.

Προτεραιότητα: MUST HAVE

- Δημιουργία πάγιων εντολών τύπου:
 - *Transfer Order* (μεταφορά μεταξύ λογαριασμών)
 - *Payment Order* (πληρωμή bill μέσω RF)
- Εκτέλεση αυτόματα στις καθορισμένες ημερομηνίες από το σύστημα της Τράπεζας.
- Καταχώρηση στοιχείων: τίτλος, περιγραφή, ποσό, ημερομηνίες έναρξης/λήξης, συχνότητα εκτέλεσης.

Προτεραιότητα: SHOULD HAVE

- Αν αποτύχουν **3 συνεχόμενες φορές**, η εντολή παραμένει ενεργή αλλά η απόπειρα ακυρώνεται.

Προτεραιότητα: COULD HAVE

- Ο Admin μπορεί να δει όλες τις ενεργές, ληγμένες ή αποτυχημένες πάγιες.

6.7 Προσομοίωση χρόνου (Time Simulation)

Η λειτουργία “Simulate Time Passing” επιτρέπει στον διαχειριστή να προσομοιώνει το πέρασμα ημερών, ώστε να ελέγχει αυτόματες διαδικασίες του συστήματος.

Προτεραιότητα: MUST HAVE

- Εκτέλεση simulation από επιλεγμένη ημερομηνία έναρξης έως ημερομηνία λήξης.
 - Κατά την προσομοίωση, το σύστημα εκτελεί **καθημερινά**:
 - τόκοι,
 - πάγιες εντολές,
 - τέλη συντήρησης.
 - Η λειτουργία simulation είναι **υποχρεωτική** για τον έλεγχο του συστήματος και για τα σενάρια δοκιμών (testing).

Προτεραιότητα: SHOULD HAVE

- Καταγραφή όλων των ενεργειών σε αρχείο καταγραφής (log file).

Προτεραιότητα: COULD HAVE

- Εμφάνιση ή εξαγωγή αναφοράς αποτελεσμάτων (report).

6.7 Audit και Αναφορές (Audit & Reporting)

Για λόγους κανονιστικής συμμόρφωσης, ασφάλειας και εσωτερικού ελέγχου, το σύστημα πρέπει να καταγράφει και να επιτρέπει αναφορές όλων των ενεργειών.

Προτεραιότητα: MUST HAVE

- Αυτόματη καταγραφή όλων των ενεργειών χρηστών και συστήματος σε **audit log**
- (ημερομηνία, χρήστης/actor, ενέργεια, αποτέλεσμα).
- Το **audit log** είναι διαθέσιμο μόνο σε διαχειριστές.
-

Προτεραιότητα: SHOULD HAVE

- Δυνατότητα εξαγωγής **reports**:
 - Συνολικές συναλλαγές ανά πελάτη ή περίοδο.
 - Αποτυχημένες πάγιες εντολές.
 - Τόκοι και τέλη ανά μήνα.
 - Οι αναφορές υποστηρίζουν μορφές εξαγωγής **CSV(Comma-Separated Values)** ή **JSON(JavaScript Object Notation)**.
 - Τα δεδομένα audit διατηρούνται για τουλάχιστον 12 μήνες για λόγους εποπτείας.

7. Μη Λειτουργικές απαιτήσεις Συστήματος (Non-Functional Requirements Systems)

Στην ενότητα αυτή καθορίζεται ο τρόπος λειτουργίας *του* συστήματος (ποιότητα, απόδοση, ασφάλεια, χρηστικότητα, διαθεσιμότητα) του e-Banking της Bank of TUC. Οι απαιτήσεις αυτές ισχύουν για όλο το σύστημα και έχουν ιεραρχηθεί επίσης με τη μεθοδολογία **MoSCoW**.

7.1 Ασφάλεια (Security)

Προτεραιότητα: **MUST HAVE**

- Ταυτοποίηση χρηστών με μοναδικό username και κωδικό πρόσβασης.
- Αποθήκευση συνθηματικών με κρυπτογράφηση (π.χ. hash).
- Κλείδωμα λογαριασμού μετά από 3 αποτυχημένες προσπάθειες login.
- Καταγραφή όλων των ενεργειών σε audit log (με ημερομηνία και actor).
- Εσωτερική πολιτική Bank of TUC για προστασία δεδομένων (GDPR compliance).

Προτεραιότητα: **SHOULD HAVE**

- Διπλός έλεγχος ταυτότητας (2FA).
- Αυτόματη αποσύνδεση μετά από 5 λεπτά αδράνειας.

7.2 Απόδοση & Διαθεσιμότητα (Performance & Availability)

Προτεραιότητα: **MUST HAVE**

- Ο μέσος χρόνος απόκρισης εντολής CLI να μην υπερβαίνει το 1 δευτερόλεπτο.
- Οι βασικές λειτουργίες να ολοκληρώνονται χωρίς σφάλμα υπό φυσιολογικό φόρτο (≤ 10 ταυτόχρονες ενέργειες).
- Η προσομοίωση χρόνου (Simulation) να εκτελείται με συνέπεια σε όλες τις ημέρες.

Προτεραιότητα: **SHOULD HAVE**

- Ανάκαμψη από σφάλματα (error recovery) χωρίς επανεκκίνηση του συστήματος.
- Εμφάνιση προειδοποίησης σε υπέρβαση χρόνου εκτέλεσης (>2 sec).

7.3 Αξιοπιστία & Ακεραιότητα Δεδομένων (Reliability & Data Integrity)

Προτεραιότητα: **MUST HAVE**

- Οι συναλλαγές να εκτελούνται **ατομικά (atomic)** – αν αποτύχει ένα βήμα, δεν αλλάζει κανένα υπόλοιπο.
- Κάθε συναλλαγή συνδέεται με Transaction ID και αντίστοιχα statements.
- Σε σφάλμα συστήματος, δεν πρέπει να υπάρχουν “ορφανές” συναλλαγές.

Προτεραιότητα: **SHOULD HAVE**

- Αυτόματος έλεγχος συνοχής balances (reconciliation) μετά από simulation.

7.4 Χρηστικότητα (Usability)

Προτεραιότητα: **MUST HAVE**

- Εύκολο, κατανοητό περιβάλλον CLI με σαφή μενού ανά ρόλο.
- Εμφάνιση επεξηγηματικών μηνυμάτων (π.χ. “ανεπαρκές υπόλοιπο”).

Προτεραιότητα: **SHOULD HAVE**

- Καθοδηγητικά μηνύματα (tooltips ή prompts).
- Συντομεύσεις πληκτρολογίου για συχνές ενέργειες (bonus).

7.5 Επεκτασιμότητα (Extensibility)

Προτεραιότητα: **MUST HAVE**

- Διαχωρισμός επιχειρησιακής λογικής (business logic) από μηχανισμό συναλλαγών (Bridge pattern).
- Δυνατότητα προσθήκης νέων τύπων συναλλαγών ή πρωτοκόλλων (SEPA, SWIFT) χωρίς επανασχεδίαση.

Προτεραιότητα: **SHOULD HAVE**

- Παράμετροι (π.χ. επιτόκιο, fees) να μπορούν να τροποποιούνται χωρίς αλλαγή κώδικα.

7.6 Συντηρησιμότητα & Καταγραφή Σφαλμάτων (Maintainability & Logging)

Προτεραιότητα: **MUST HAVE**

- Καταγραφή όλων των λαθών με ημερομηνία και περιγραφή.
- Ευδιάκριτα μηνύματα σφάλματος προς τον χρήστη.

Προτεραιότητα: **SHOULD HAVE**

- Δυνατότητα εξαγωγής logs σε αρχείο για ανάλυση.

7.7 Κανονιστική Συμμόρφωση (Compliance)

Προτεραιότητα: **MUST HAVE**

- Τήρηση κανονισμών GDPR (προστασία προσωπικών δεδομένων).
- Διατήρηση audit logs για τουλάχιστον 12 μήνες.
- Πρόσβαση στα logs μόνο από εξουσιοδοτημένους διαχειριστές.

Προτεραιότητα: **SHOULD HAVE**

- Εφαρμογή πολιτικών Bank of TUC για περιοδικό έλεγχο ασφάλειας.

7.8 Φορητότητα & Περιβάλλον Εκτέλεσης (Portability)

Προτεραιότητα: **MUST HAVE**

- Το σύστημα πρέπει να εκτελείται σε περιβάλλον Windows, Linux ή MacOS χωρίς αλλαγές στον κώδικα.

Προτεραιότητα: **COULD HAVE**

- Δυνατότητα επέκτασης σε περιβάλλον web ή mobile σε μελλοντική φάση.

8 . Σύνοψη

Στην τελευταία ενότητα παρουσιάζονται συγκεντρωτικά οι λειτουργικές και μη λειτουργικές απαιτήσεις του συστήματος e-Banking της **Bank of TUC**, ταξινομημένες βάσει της προτεραιότητας **MoSCoW** (*Must, Should, Could*) αλλά και ένα τελικό συμπέρασμα για όλη την διεξαγωγή της έρευνας.

8.1 Πίνακας Λειτουργικών Απαιτήσεων

Ενότητα	Must Have	Should Have	Could Have
Διαχείριση Χρηστών (User Management)	Login με username & password, διάκριση ρόλων (Individual, Company, Admin)	Ο Admin μπορεί να δει/διαχειρίζεται στοιχεία πελατών	–
Διαχείριση Λογαριασμών (Accounts)	Δημιουργία IBAN, απόδοση τόκων, χρέωση τελών εταιρικών λογαριασμών	Συνδικαιούχοι, ενεργοποίηση/απενεργοποίηση λογαριασμών	–
Συναλλαγές (Transactions)	Καταθέσεις, αναλήψεις, μεταφορές, πληρωμές μέσω RF, statements	Έλεγχος εγκυρότητας ποσών και υπολοίπων	–

Κινήσεις Λογαριασμών (Statements)	Καταγραφή κινήσεων: ημερομηνία, ποσό, τύπος, υπόλοιπο, αιτιολογία	Φιλτράρισμα / εξαγωγή κινήσεων, πρόσβαση admin	–
Λογαριασμοί Πληρωμών (Bills)	Έκδοση & πληρωμή Bills με RF code	Έλεγχος ληγμένου ή διπλού RF code	Μαζική διαχείριση bills από admin
Πάγιες Εντολές (Standing Orders)	Αυτόματη εκτέλεση μεταφορών/πληρωμών , πολιτική 3 αποτυχιών	–	Προβολή ενεργών ή αποτυχημένων εντολών από admin
Προσομοίωση Χρόνου (Time Simulation)	Εκτέλεση simulation (τόκοι, πάγιες, τέλη)	Καταγραφή ενεργειών σε log	Εξαγωγή αναφοράς αποτελεσμάτων
Audit & Αναφορές (Audit & Reporting)	Καταγραφή ενεργειών χρηστών και συστήματος	Εξαγωγή reports, διατήρηση audit data 12 μηνών	–

8.2 Πίνακας Μη Λειτουργικών Απαιτήσεων

Κατηγορία	Must Have	Should Have	Could Have
Ασφάλεια (Security)	Κρυπτογράφηση συνθηματικών, 3 αποτυχημένα login → κλείδωμα, GDPR συμμόρφωση	2FA, αυτόματη αποσύνδεση μετά από 5'	–
Απόδοση & Διαθεσιμότητα (Performance & Availability)	CLI εντολές ≤1 sec, σταθερή λειτουργία έως 10 χρήστες	Ανάκαμψη από σφάλματα χωρίς επανεκκίνηση	–
Αξιοπιστία & Ακεραιότητα (Reliability & Data Integrity)	Συναλλαγές atomic, χωρίς “ορφανές” εγγραφές	Έλεγχος συνοχής balances μετά από simulation	–
Χρηστικότητα (Usability)	Καθαρό CLI, σαφή μηνύματα σφαλμάτων	Prompts, συντομεύσεις ενεργειών	–

Επεκτασιμότητα (Extensibility)	Bridge pattern, υποστήριξη νέων πρωτοκόλλων (SEPA/SWIFT)	Αλλαγή παραμέτρων χωρίς αλλαγή κώδικα	–
Συντηρησιμότητα & Logging	Καταγραφή λαθών και logs	Εξαγωγή logs για ανάλυση	–
Κανονιστική Συμμόρφωση (Compliance)	GDPR, διατήρηση logs 12 μηνών	Περιοδικός έλεγχος ασφάλειας	–
Φορητότητα (Portability)	Εκτέλεση σε Windows, Linux, MacOS	–	Μελλοντική επέκταση σε web ή mobile

8.3 Συμπεράσματα Έρευνας

Η ανάλυση και σχεδίαση του συστήματος e-Banking για την Bank of TUC μας έδωσε μια ολοκληρωμένη εικόνα για το πώς οι πελάτες και οι διαχειριστές μπορούν να χρησιμοποιούν την πλατφόρμα με ασφάλεια και ευκολία. Καθορίσαμε τους βασικούς ρόλους, τις σημαντικές οντότητες και τις λειτουργίες για καθημερινές συναλλαγές, δίνοντας έμφαση στην αξιοπιστία, την ασφάλεια και την ευελιξία. Παράλληλα, η ανάπτυξη του συστήματος υπόσχεται μείωση κόστους λειτουργίας μέσω αυτοματισμών και λιγότερων χειροκίνητων διαδικασιών, ενώ δημιουργεί τα θεμέλια για μελλοντικές επεκτάσεις χωρίς σημαντικές πρόσθετες δαπάνες. Το αποτέλεσμα είναι μια φιλική και αποτελεσματική εμπειρία για όλους τους χρήστες (ιδιώτες και επιχειρήσεις), με έλεγχο κόστους και βελτίωση της αποδοτικότητας της τράπεζας.

Στη Φάση II μας ζητηθηκε ο σχεδιασμός του υπο ανάπτυξη σύστημα e-Banking συμμεχοντας 4 βασικοί ρόλοι.

1. Individual Customer (Ιδιώτης Πελάτης)

Ο Ιδιώτης Πελάτης μπορεί να εκτελεί τις ακόλουθες ενέργειες:

- Διαχείριση Λογαριασμών
 - Προβολή Λογαριασμών
 - Προβολή Κινήσεων (Statements)
- Εκτέλεση Συναλλαγών
 - Κατάθεση
 - Ανάληψη
 - Μεταφορά Χρημάτων
 - Πληρωμή Λογαριασμού μέσω κωδικού RF
- Διαχείριση Πάγιας Εντολών
 - Δημιουργία Πάγιας Εντολής
 - Ακύρωση Πάγιας Εντολής

2. Company Customer (Επιχείρηση)

Η Επιχείρηση θα έχει πρόσβαση στον επιχειρηματικό της λογαριασμό και θα μπορεί:

- Διαχείριση Λογαριασμών
 - Προβολή Λογαριασμού
 - Προβολή Κινήσεων
- Εκτέλεση Συναλλαγών
 - Μεταφορά Χρημάτων

➤ Διαχείριση Λογαριασμών Πληρωμής (Bills)

- Έκδοση Λογαριασμού Πληρωμής (RF)
- Προβολή Εκδοθέντων Λογαριασμών
- Προβολή Εξοφλημένων Λογαριασμών

➤ Διαχείριση Πάγιων Εντολών

- Δημιουργία Πάγιας Εντολής
 - Ακύρωση Πάγιας Εντολής
-

3. Admin (Διαχειριστής)

Ο Διαχειριστής διαθέτει εκτεταμένες δυνατότητες εποπτείας και συστημικού ελέγχου:

➤ Διαχείριση Λογαριασμών

- Προβολή Όλων των Πελατών
- Προβολή Όλων των Λογαριασμών
- Προβολή Κινήσεων Οποιοδήποτε Λογαριασμού

➤ Λειτουργία (Simulate Time Passing)

- Αυτόματη Εκτέλεση Πάγιων Εντολών
- Υπολογισμός Τόκων
- Χρέωση Τελών Συντήρησης

➤ Έλεγχος Συστήματος

- Προβολή Audit Log
 - Χειροκίνητη Πληρωμή Λογαριασμού
-

4. Bank System (Σύστημα Τράπεζας)

Το Τραπεζικό Σύστημα λειτουργεί ως αυτόματος actor και εκτελεί:

➤ Αυτόματες Λειτουργίες Συναλλαγών

- Αυτόματη Εκτέλεση Πάγιων Εντολών

➤ Υπολογιστικές Λειτουργίες

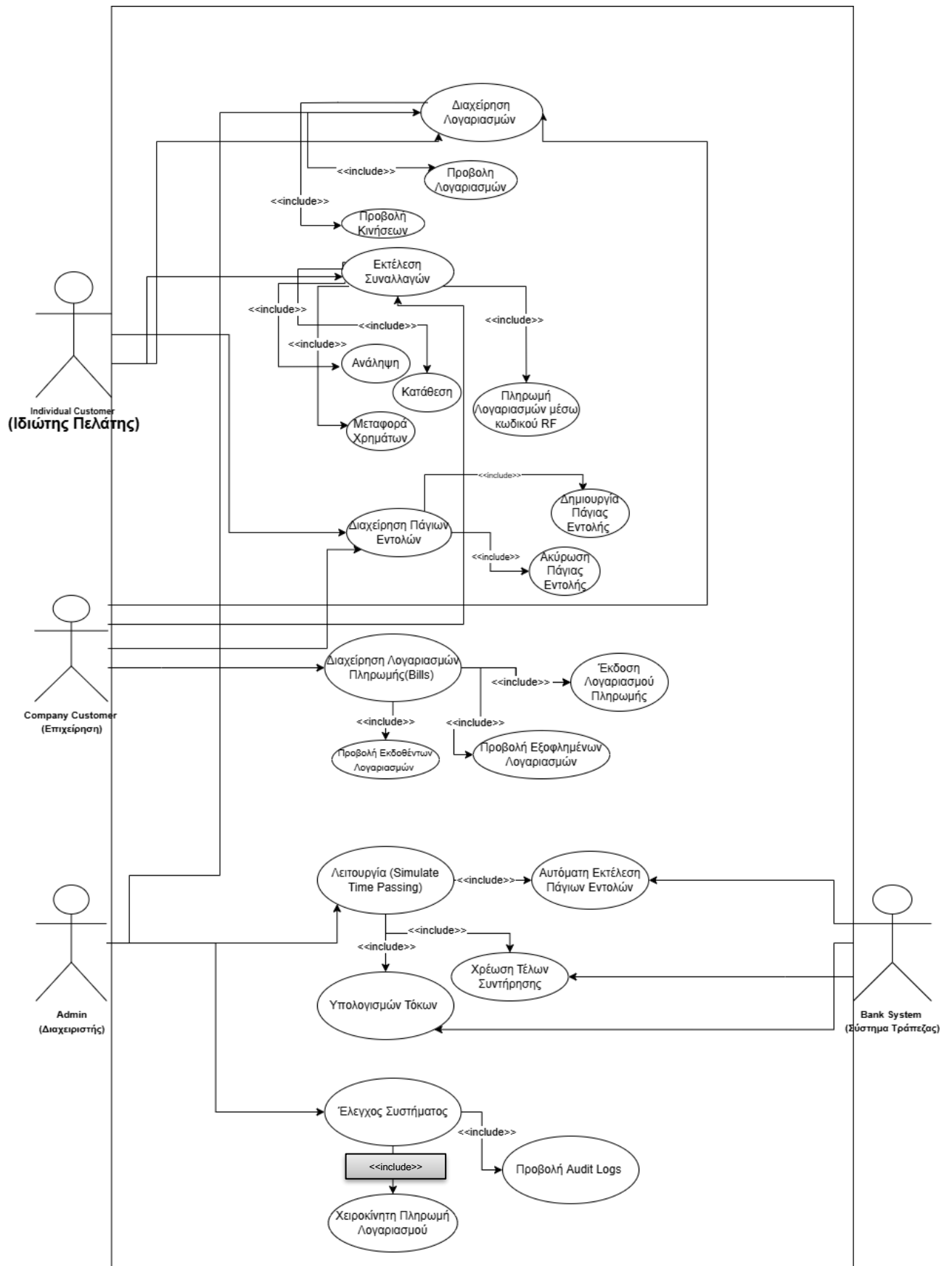
- Υπολογισμός Τόκων
- Καταχώριση Τόκων σε Λογαριασμούς

➤ Διαχείριση Τελών

- Μηνιαία Χρέωση Τελών Συντήρησης

USE CASE DIAGRAM

Παρακάτω βλέπουμε πως πραγματοποιήθηκε η ανάλυση των λειτουργικών απαιτήσεων του συστήματος E-banking , μέσω ενός Use Case Diagram. Το διάγραμμα αυτό αποτυπώνει τους βασικούς εξωτερικούς χρήστες (actors) του συστήματος (Ιδιώτη Πελάτη, Επιχείρηση, Διαχειριστή (Admin), Τραπεζικό Σύστημα) καθώς και τις κύριες λειτουργίες που αυτοί μπορούν να εκτελέσουν. Η μοντελοποίηση που έγινε συμβάλλει στην κατανόηση συμπεριφοράς του συστήματος ομαδοποιώντας τις λειτουργίες σε επιμέρους ενότητες, όπως Διαχείριση Λογαριασμών, Εκτέλεση Συναλλαγών, Διαχείριση Πάγιων Εντολών και Λειτουργίες Συστήματος. Μέσω τις σχέσεις include όπως θα δούμε παρακάτω αποτυπώνονται οι υπό-λειτουργίες που εκτελούνται. Το τελικό διάγραμμα που σχεδιάσαμε παρουσιάζει συνολικά την σχέση των χρηστών με το σύστημα καθώς και τις αυτόματες διαδικασίες που εκτελούνται από το Τραπεζικό Σύστημα.



1. Διαχείριση Λογαριασμών

Η περίπτωση χρήσης «Διαχείριση Λογαριασμών» επιτρέπει στον Ιδιώτη Πελάτη, στην Επιχείρηση και στον Διαχειριστή (Admin) να προβάλλουν βασικές πληροφορίες σχετικά με τους τραπεζικούς λογαριασμούς ενός χρήστη. Περιλαμβάνει λειτουργίες για την προβολή όλων των διαθέσιμων λογαριασμών, καθώς και την εμφάνιση αναλυτικών κινήσεων για κάθε λογαριασμό. Μέσω των υπο-περιπτώσεων χρήσης «Προβολή Λογαριασμών» και «Προβολή Κινήσεων», οι χρήστες αποκτούν πλήρη εικόνα των υπολοίπων, των συναλλαγών και της γενικής δραστηριότητας σε κάθε λογαριασμό. Η περίπτωση χρήσης αυτή είναι πολύ σημαντική για την πλοήγηση στο σύστημα e-Banking και αποτελεί προϋπόθεση για άλλες λειτουργίες, όπως οι πληρωμές και οι μεταφορές χρημάτων.

2. Εκτέλεση Συναλλαγών

Η περίπτωση χρήσης «Εκτέλεση Συναλλαγών» επιτρέπει σε έναν Ιδιώτη Πελάτη να πραγματοποιεί όλες τις βασικές χρηματικές συναλλαγές. Η λειτουργία αυτή περιλαμβάνει τα υπο-Use Cases «Κατάθεση», «Ανάληψη», «Μεταφορά Χρημάτων» και «Πληρωμή Λογαριασμού μέσω Κωδικού RF». Μέσω αυτών, ο χρήστης μπορεί να μεταφέρει ποσά μεταξύ λογαριασμών, να εξοφλεί λογαριασμούς, να καταθέτει χρήματα στον λογαριασμό του ή να πραγματοποιεί αναλήψεις. Η εκτέλεση συναλλαγών είναι η κεντρική λειτουργία του συστήματος και απαιτείται και έλεγχος επάρκειας υπολοίπου, έλεγχο εγκυρότητας RF και ενημέρωση του ιστορικού κινήσεων.

3. Διαχείριση Πάγιων Εντολών

Η περίπτωση χρήσης «Διαχείριση Πάγιων Εντολών» επιτρέπει στον Ιδιώτη Πελάτη να δημιουργεί και να διαχειρίζεται εντολές που εκτελούνται περιοδικά και αυτόματα από το σύστημα. Η λειτουργία περιλαμβάνει τα υπο-Use Cases «Δημιουργία Πάγιας Εντολής» και «Ακύρωση Πάγιας Εντολής». Οι πάγιες εντολές μπορούν να αφορούν μεταφορές χρημάτων ή πληρωμές λογαριασμών, προσφέροντας αυτοματοποίηση και ευκολία. Το σύστημα ελέγχει τον προγραμματισμό, τις ημερομηνίες εκτέλεσης και την επάρκεια υπολοίπου, ενώ καταγράφει τυχόν αποτυχίες.

4. Διαχείριση Λογαριασμών Πληρωμής (Bills)

Η περίπτωση χρήσης «Διαχείριση Λογαριασμών Πληρωμής (Bills)» χρησιμοποιείται αποκλειστικά από τους Εταιρικούς Πελάτες. Επιτρέπει την έκδοση νέων λογαριασμών πληρωμής με μοναδικό κωδικό RF προς πελάτες της επιχείρησης. Περιλαμβάνει τρεις υπο-λειτουργίες: «Έκδοση Λογαριασμού Πληρωμής», «Προβολή Εκδοθέντων Λογαριασμών» και «Προβολή Εξοφλημένων Λογαριασμών». Η λειτουργία αυτή διευκολύνει την παρακολούθηση εισπραξέων και αυτοματοποιεί την ταυτοποίηση πληρωμών μέσω του κωδικού RF.

5. Λειτουργία Simulate Time Passing (Admin)

Η περίπτωση χρήσης «Λειτουργία Simulate Time Passing» επιτρέπει στον Διαχειριστή να προσομοιώνει την ημερήσια λειτουργία του συστήματος. Με την ενεργοποίηση της διαδικασίας αυτής εκτελούνται αυτόματα λειτουργίες που σχετίζονται με τον χρόνο, όπως η «Αυτόματη Εκτέλεση Πάγιων Εντολών», ο «Υπολογισμός Τόκων» και η «Χρέωση Τελών Συντήρησης». Η χρήση του simulate time χρησιμεύει σε περιβάλλοντα δοκιμών ή σε περιπτώσεις που απαιτείται η εκτέλεση προγραμματισμένων λειτουργιών χωρίς φυσική πάροδο ημερών.

6. Έλεγχος Συστήματος (Admin)

Η περίπτωση χρήσης «Έλεγχος Συστήματος» επιτρέπει στον Διαχειριστή να παρακολουθεί την εσωτερική δραστηριότητα και λειτουργικότητα της πλατφόρμας. Περιλαμβάνει τα υπο-Use Cases «Προβολή Audit Logs» και «Χειροκίνητη Πληρωμή Λογαριασμού». Μέσω των Audit Logs ο διαχειριστής έχει πρόσβαση σε καταγεγραμμένα συμβάντα, όπως συνδέσεις χρηστών, εκτελεσμένες συναλλαγές και αποτυχίες συστήματος. Η δυνατότητα χειροκίνητης πληρωμής παρέχει έναν τρόπο επίλυσης προβλημάτων ή διόρθωσης συναλλαγών σε ειδικές περιπτώσεις.

7. Αυτόματες Λειτουργίες Τραπεζικού Συστήματος

Οι αυτόματες λειτουργίες εκτελούνται από τον actor «Bank System» χωρίς άμεση ανθρώπινη παρέμβαση. Περιλαμβάνουν την «Αυτόματη Εκτέλεση Πάγιων Εντολών», τον «Υπολογισμό Τόκων» και τη «Χρέωση Τελών Συντήρησης». Οι διαδικασίες αυτές ενεργοποιούνται αυτόματα σε προκαθορισμένες ημερομηνίες ή μέσω της λειτουργίας simulate time. Αποτελούν κρίσιμο μέρος της απρόσκοπτης λειτουργίας του συστήματος e-banking και διασφαλίζουν τη σωστή ενημέρωση λογαριασμών και υπολοίπων.

FR CODES

Παρακάτω παρουσιάζονται κωδικοί FR χρησιμοποιούνται σε όλα τα Use Case Scenarios & GUI Storyboards.

- **USER MANAGEMENT**

Κωδικός	Περιγραφή
FR-UM-01	Login με username και password
FR-UM-02	Διάκριση ρόλων (Individual, Company, Admin)
FR-UM-03	Προβολή / διαχείριση στοιχείων πελατών από Admin

- **ACCOUNTS**

Κωδικός	Περιγραφή
FR-AC-01	Προβολή λογαριασμών (IBAN, υπόλοιπο, επιτόκιο, κατάσταση)
FR-AC-02	Δημιουργία τραπεζικού λογαριασμού (IBAN)
FR-AC-03	Μηνιαία χρέωση τέλους συντήρησης (Business Accounts)
FR-AC-04	Υπολογισμός & απόδοση τόκων στο τέλος του μήνα

Κωδικός	Περιγραφή
FR-AC-05	Ενεργοποίηση/Απενεργοποίηση λογαριασμών από Admin

- TRANSACTIONS

Κωδικός	Περιγραφή
FR-TX-01	Κατάθεση χρημάτων
FR-TX-02	Ανάληψη χρημάτων
FR-TX-03	Μεταφορά χρημάτων (Intra-bank transfer)
FR-TX-04	Πληρωμή Bill μέσω RF code
FR-TX-05	Έλεγχος εγκυρότητας ποσού & επαρκούς υπολοίπου
FR-TX-06	Interest Credit (αυτόματη πίστωση τόκων)
FR-TX-07	Fee Debit (αυτόματη χρέωση τελών)

- STATEMENTS

Κωδικός	Περιγραφή
FR-ST-01	Δημιουργία statement για κάθε συναλλαγή
FR-ST-02	Προβολή statements (ημ/νία, τύπος, ποσό, υπόλοιπο, αιτιολογία)
FR-ST-03	Φιλτράρισμα statements ανά περίοδο
FR-ST-04	Admin access σε statements όλων των πελατών

BILLS

Κωδικός	Περιγραφή
FR-BL-01	Έκδοση Bill με RF code από εταιρεία
FR-BL-02	Διαχείριση κατάστασης Bill (ενεργός/εξοφλημένος/ληγμένος)
FR-BL-03	Πληρωμή ενεργού Bill μέσω RF
FR-BL-04	Έλεγχος ληγμένου / ήδη εξοφλημένου RF

- **STANDING ORDERS**

Κωδικός	Περιγραφή
FR-SO-01	Δημιουργία Transfer Standing Order
FR-SO-02	Δημιουργία Payment Standing Order (RF)
FR-SO-03	Αυτόματη εκτέλεση πάγιας εντολής
FR-SO-04	Πολιτική 3 αποτυχιών για πάγιες
FR-SO-06	Ακύρωση πάγιας εντολής
FR-SO-05	Προβολή λίστας πάγιων εντολών

- **TIME SIMULATION**

Κωδικός	Περιγραφή
FR-SM-01	Εκτέλεση προσομοίωσης ημερών (Simulate Time Passing)
FR-SM-02	Καθημερινές αυτόματες λειτουργίες (τόκοι, πάγιες, τέλη)
FR-SM-03	Καταγραφή σε log file

- **AUDIT REPORTING**

Κωδικός	Περιγραφή
FR-AU-01	Audit log όλων των ενεργειών
FR-AU-02	Πρόσβαση audit μόνο από Admin
FR-AU-03	Εξαγωγή αναφορών (transactions / failed standing orders / fees)

USE CASE SCENARIOS

Στην ενότητα αυτή παρουσιάζονται αναλυτικά σενάρια για βασικές περιπτώσεις χρήσης του Individual Customer (Ιδιώτη Πελάτη) στο σύστημα e-Banking της Bank of TUC. Κάθε σενάριο περιγράφεται με όρους προϋποθέσεων, βασικής ροής, εναλλακτικών ροών και τελικών καταστάσεων, και συνοδεύεται από αναφορά στις αντίστοιχες Λειτουργικές Απαιτήσεις (Functional Requirements - FRs) όπως αυτές έχουν οριστεί στην ενότητα των

απαιτήσεων.

UC-01: Προβολή Λογαριασμών στον Ιδιώτη

Primary Actor: Individual Customer (Ιδιώτης Πελάτης)

Στόχος: Ο πελάτης να δει όλους τους λογαριασμούς του και τα τρέχοντα υπόλοιπα.

Preconditions:

- Ο πελάτης έχει κάνει επιτυχές login στο e-banking.
- Ο πελάτης έχει τουλάχιστον έναν ενεργό λογαριασμό.

Postconditions:

- Έχει εμφανιστεί στον χρήστη η λίστα των λογαριασμών με τα βασικά στοιχεία (IBAN, τύπος, υπόλοιπο).

Basic Flow

1. Ο χρήστης, από την αρχική οθόνη (dashboard), επιλέγει “Λογαριασμοί” ή “Προβολή Λογαριασμών”.
2. Το σύστημα ανακτά τη λίστα με όλους τους ενεργούς λογαριασμούς του πελάτη.
3. Το σύστημα εμφανίζει σε πίνακα τους λογαριασμούς με IBAN, τύπο λογαριασμού και διαθέσιμο υπόλοιπο.
4. Ο χρήστης επιλέγει έναν λογαριασμό για περισσότερες λεπτομέρειες (π.χ. κινήσεις).

Alternative Flows

- **A1 - Δεν υπάρχουν ενεργοί λογαριασμοί**
 1. Αν ο πελάτης δεν έχει ενεργούς λογαριασμούς, το σύστημα εμφανίζει μήνυμα “Δεν υπάρχουν ενεργοί λογαριασμοί”.
 2. Η ροή τερματίζει.

Σχετικές FRs

- **FR-AC-01** – Προβολή λογαριασμών (IBAN, υπόλοιπο, επιτόκιο, status)
- **FR-UM-01** – Login (προϋπόθεση)
- **FR-UM-02** – Διαχείριση ρόλων (επιτρέπει πρόσβαση Ιδιώτη)

UC-02: Μεταφορά Χρημάτων στον Ιδιώτη

Primary Actor: Individual Customer

Στόχος: Ο πελάτης μεταφέρει χρήματα από έναν δικό του ή εξουσιοδοτημένο λογαριασμό σε άλλον λογαριασμό της Bank of TUC.

Preconditions:

- Ο πελάτης είναι συνδεδεμένος στο σύστημα (login).
- Ο πελάτης έχει πρόσβαση στον λογαριασμό προέλευσης.

Postconditions:

- Αν η συναλλαγή είναι επιτυχής, έχουν ενημερωθεί τα υπόλοιπα των δύο λογαριασμών και έχει δημιουργηθεί εγγραφή συναλλαγής/κινήσεων.
- Αν απορριφθεί, δεν γίνεται καμία αλλαγή στα υπόλοιπα.

Basic Flow

1. Ο χρήστης από το dashboard επιλέγει “Μεταφορά Χρημάτων”.
2. Το σύστημα εμφανίζει φόρμα με πεδία: From IBAN, To IBAN, ποσό, αιτιολογία.
3. Ο χρήστης συμπληρώνει όλα τα πεδία και πατάει “Επιβεβαίωση”.
4. Το σύστημα ελέγχει ότι και τα δύο IBAN ανήκουν στην Bank of TUC και είναι ενεργοί λογαριασμοί.
5. Το σύστημα ελέγχει ότι ο λογαριασμός προέλευσης έχει επαρκές διαθέσιμο υπόλοιπο.
6. Το σύστημα δημιουργεί τη συναλλαγή μεταφοράς.
7. Το σύστημα μειώνει το υπόλοιπο του From IBAN και αυξάνει το υπόλοιπο του To IBAN (atomic ενημέρωση).
8. Το σύστημα καταγράφει τις αντίστοιχες κινήσεις (statements) και εμφανίζει μήνυμα επιτυχίας.

Alternative Flows

- **A1 - Ανεπαρκές υπόλοιπο**
 1. Στο βήμα 5, αν το υπόλοιπο δεν επαρκεί, το σύστημα ακυρώνει τη συναλλαγή.
 2. Εμφανίζει μήνυμα “Ανεπαρκές υπόλοιπο”.
 3. Ο χρήστης μπορεί να διορθώσει το ποσό ή να ακυρώσει τη διαδικασία.
- **A2 - Μη έγκυρο ή ανενεργό IBAN**
 1. Στο βήμα 4, αν κάποιο IBAN είναι λάθος ή ανενεργό, το σύστημα εμφανίζει μήνυμα σφάλματος.
 2. Ο χρήστης επιστρέφει στη φόρμα και διορθώνει τα στοιχεία.

Σχετικές FRs

- **FR-TX-03** – Μεταφορά χρημάτων (Intra-bank transfer)
- **FR-TX-02** – Ανάληψη χρημάτων (χρέωση From IBAN ως withdrawal)
- **FR-TX-01** – Κατάθεση χρημάτων (πίστωση To IBAN ως deposit)
- **FR-TX-05** – Έλεγχος εγκυρότητας ποσού & επαρκούς υπολοίπου
- **FR-ST-01** – Δημιουργία statements για τη συναλλαγή
- **FR-ST-02** – Προβολή statements (έμμεσα χρησιμοποιείται)
- **FR-AC-01** – Προβολή λογαριασμών (για επιλογή IBAN)
- **FR-UM-01** – Login (προϋπόθεση)

Primary Actor: Individual Customer

Στόχος: Ο πελάτης να εξοφλήσει έναν λογαριασμό (Bill) χρησιμοποιώντας τον κωδικό RF.

Preconditions:

- Ο πελάτης είναι συνδεδεμένος.
- Υπάρχει τουλάχιστον ένας ενεργός λογαριασμός με επαρκές υπόλοιπο.

Postconditions:

- Αν η πληρωμή γίνει, ο λογαριασμός πληρωμής (Bill) ενημερώνεται σε “Εξοφλημένος” και τα υπόλοιπα ενημερώνονται.
- Αν απορριφθεί, δεν αλλάζει κανένα υπόλοιπο ή status.

Basic Flow

1. Ο χρήστης από το μενού επιλέγει “Πληρωμή Λογαριασμών μέσω κωδικού RF”.
2. Το σύστημα εμφανίζει φόρμα με πεδία: RF κωδικός, λογαριασμός χρέωσης (From IBAN).
3. Ο χρήστης εισάγει τον RF κωδικό και επιλέγει τον λογαριασμό χρέωσης.
4. Το σύστημα αναζητά τον λογαριασμό πληρωμής (Bill) που αντιστοιχεί στον κωδικό RF.
5. Το σύστημα εμφανίζει τα στοιχεία του Bill (ποσό, περιγραφή, ημερομηνία λήξης) και ζητά επιβεβαίωση.
6. Ο χρήστης επιβεβαιώνει την πληρωμή.
7. Το σύστημα ελέγχει επάρκεια υπολοίπου στον λογαριασμό χρέωσης.
8. Το σύστημα χρεώνει το ποσό στον λογαριασμό του πελάτη και πιστώνει τον λογαριασμό της επιχείρησης.
9. Το σύστημα ενημερώνει τον Bill σε κατάσταση “Εξοφλημένος” και καταγράφει τις κινήσεις.
10. Το σύστημα εμφανίζει μήνυμα επιτυχούς πληρωμής.

Alternative Flows

- **A1 - Άκυρος RF κωδικός**
 1. Στο βήμα 4, αν δεν βρεθεί Bill για τον RF κωδικό, το σύστημα εμφανίζει μήνυμα “Μη έγκυρος κωδικός RF”.
 2. Ο χρήστης μπορεί να ξαναπροσπαθήσει ή να ακυρώσει.
- **A2 - Το Bill είναι ήδη εξοφλημένο ή ληγμένο**
 1. Αν ο Bill έχει status “Εξοφλημένος” ή έχει λήξει, το σύστημα ενημερώνει τον χρήστη ότι η πληρωμή δεν μπορεί να γίνει.
 2. Η ροή τερματίζει χωρίς χρηματική κίνηση.
- **A3 - Ανεπαρκές υπόλοιπο**
 1. Στο βήμα 7, αν το υπόλοιπο είναι ανεπαρκές, το σύστημα ακυρώνει την πληρωμή.
 2. Εμφανίζει μήνυμα “Ανεπαρκές υπόλοιπο”.

Σχετικές FRs

- **FR-TX-04** - Πληρωμή Bill μέσω RF code
- **FR-BL-02** - Διαχείριση κατάστασης Bill (ενεργός/εξοφλημένος/ληγμένος)
- **FR-BL-03** - Πληρωμή ενεργού Bill μέσω RF

- **FR-BL-04** – Έλεγχος ληγμένου / ήδη εξοφλημένου RF
 - **FR-TX-05** – Έλεγχος εγκυρότητας ποσού & επαρκούς υπολοίπου
 - **FR-ST-01** – Δημιουργία statements
 - **FR-AC-01** – Προβολή λογαριασμών για επιλογή λογαριασμού χρέωσης
 - **FR-UM-01** – Login (προϋπόθεση)
-

UC-04: Δημιουργία Πάγιας Εντολής στον Ιδιώτη

Primary Actor: Individual Customer

Στόχος: Ο πελάτης να δημιουργήσει μια πάγια εντολή (Standing Order) για επαναλαμβανόμενη πληρωμή ή μεταφορά.

Preconditions:

- Ο πελάτης είναι συνδεδεμένος.
- Υπάρχει τουλάχιστον ένας ενεργός λογαριασμός χρέωσης.

Postconditions:

- Μια νέα ενεργή πάγια εντολή έχει δημιουργηθεί και αποθηκευτεί με τα σωστά στοιχεία (frequency, ποσό κ.λπ.).

Basic Flow

1. Ο χρήστης από το μενού επιλέγει “Διαχείριση Πάγιων Εντολών”.
2. Το σύστημα εμφανίζει τη λίστα των υπαρχουσών πάγιων εντολών και επιλογή “Νέα Πάγια Εντολή”.
3. Ο χρήστης επιλέγει “Νέα Πάγια Εντολή”.
4. Το σύστημα εμφανίζει φόρμα με πεδία: είδος εντολής (μεταφορά/πληρωμή), λογαριασμός χρέωσης, λογαριασμός/RF προορισμού, ποσό ή μέγιστο ποσό, συχνότητα (π.χ. μηνιαία), ημερομηνία έναρξης, προαιρετικά ημερομηνία λήξης ή αριθμός επαναλήψεων.
5. Ο χρήστης συμπληρώνει τα στοιχεία και πατάει “Αποθήκευση”.
6. Το σύστημα ελέγχει ότι όλα τα υποχρεωτικά πεδία είναι συμπληρωμένα και έγκυρα.
7. Το σύστημα δημιουργεί την πάγια εντολή με status “Ενεργή”.
8. Το σύστημα εμφανίζει μήνυμα επιτυχίας και ενημερωμένη λίστα πάγιων εντολών.

Alternative Flows

- **A1 - Ελλιπή ή μη έγκυρα στοιχεία**
 1. Στο βήμα 6, αν κάποιο πεδίο λείπει ή έχει λάθος μορφή (π.χ. αρνητικό ποσό), το σύστημα εμφανίζει μήνυμα λάθους.
 2. Επισημαίνει τα πεδία προς διόρθωση.
 3. Ο χρήστης διορθώνει και επιστρέφει στο βήμα 5.

Σχετικές FRs

- **FR-SO-01** - Δημιουργία Transfer Standing Order
- **FR-SO-02** - Δημιουργία Payment Standing Order (RF)
- **FR-SO-04** - Πολιτική 3 αποτυχημένων πάγιων
- **FR-SO-05** - Προβολή λίστας πάγιων εντολών
- **FR-SO-06** - Ακύρωση πάγιας εντολής (διαχειριστική δυνατότητα ενότητας Πάγιων Εντολών)
- **FR-AC-01** - Προβολή λογαριασμών για επιλογή λογαριασμού χρέωσης
- **FR-BL-02** - (Μόνο αν η πάγια αφορά πληρωμές RF)
- **FR-UM-01** - Login (προϋπόθεση)

UC-05: Έκδοση Λογαριασμών Πληρωμής στον Company Customer

Primary Actor: Company Customer (Επιχείρηση)

Στόχος: Η επιχείρηση να εκδώσει νέους λογαριασμούς πληρωμής (Bills) προς τους πελάτες της.

Preconditions:

- Η επιχείρηση είναι συνδεδεμένη στο e-banking ως Company Customer.
- Διαθέτει ενεργό business account.

Postconditions:

- Έχουν δημιουργηθεί νέοι λογαριασμοί πληρωμής με μοναδικό RF κωδικό και συσχέτιση με την επιχείρηση.

Basic Flow

1. Ο χρήστης (εταιρεία) επιλέγει από το μενού “Διαχείριση Λογαριασμών Πληρωμής (Bills)”.
2. Το σύστημα εμφανίζει λίστα με υπάρχοντες Bills και επιλογή “Έκδοση Νέου Λογαριασμού Πληρωμής”.
3. Η επιχείρηση επιλέγει “Έκδοση Νέου Λογαριασμού Πληρωμής”.
4. Το σύστημα εμφανίζει φόρμα με πεδία: ποσό, περιγραφή, ημερομηνία έκδοσης, ημερομηνία λήξης, στοιχεία πελάτη (π.χ. κωδικός πελάτη).
5. Ο χρήστης συμπληρώνει τα στοιχεία και πατάει “Έκδοση”.
6. Το σύστημα δημιουργεί νέο Bill με:
 - ο μοναδικό RF κωδικό,
 - ο status “Ενεργός”,
 - ο συσχέτιση με το business account της επιχείρησης.
7. Το σύστημα εμφανίζει επιβεβαίωση με τον RF κωδικό και τον αποθηκεύει στη λίστα Bills.

Alternative Flows

- **A1 - Ελλιπή ή λάθος στοιχεία λογαριασμού πληρωμής**
 1. Στο βήμα 6, αν κάποιο υποχρεωτικό πεδίο λείπει ή είναι μη έγκυρο (π.χ. ποσό ≤ 0), το σύστημα ενημερώνει την επιχείρηση και δεν δημιουργεί Bill.

2. Ο χρήστης διορθώνει τα στοιχεία και επιστρέφει στο βήμα 5.

Σχετικές FRs

- **FR-BL-01** - Έκδοση Bill με RF code
- **FR-BL-02** - Διαχείριση κατάστασης Bill
- **FR-UM-02** - Διαχείριση ρόλων (Company)
- **FR-AC-02** - Δημιουργία λογαριασμού πληρωμής (αν το χρησιμοποιείτε ως βάση για Bills)
- **FR-ST-01** - Δημιουργία statements (αν υπάρχει άμεση χρέωση)

UC-06: Προβολή Εξοφλημένων Λογαριασμών Πληρωμής στον Company Costumer

Primary Actor: Company Customer

Στόχος: Η επιχείρηση να δει ποιους λογαριασμούς πληρωμής έχουν εξοφλήσει οι πελάτες.

Preconditions:

- Η επιχείρηση είναι συνδεδεμένη στο e-banking.
- Υπάρχουν καταχωρημένοι λογαριασμοί πληρωμής για την επιχείρηση.

Postconditions:

- Εμφανίζεται στον χρήστη λίστα με όλους τους Bills σε κατάσταση “Εξοφλημένος”, πιθανώς με φίλτρα ημερομηνίας.

Basic Flow

1. Ο χρήστης (εταιρεία) επιλέγει από το μενού “Προβολή Εξοφλημένων Λογαριασμών”.
2. Το σύστημα ζητά τυχόν φίλτρα (π.χ. χρονικό διάστημα, ποσό, πελάτης).
3. Ο χρήστης επιλέγει τα επιθυμητά φίλτρα (προαιρετικά) και επιβεβαιώνει.
4. Το σύστημα αναζητά όλους τους λογαριασμούς πληρωμής της επιχείρησης με status “Εξοφλημένος” που ταιριάζουν στα φίλτρα.
5. Το σύστημα εμφανίζει τη λίστα με βασικές πληροφορίες (RF, ποσό, ημερομηνία εξόφλησης, πελάτης).

Alternative Flows

- **A1 - Δεν βρέθηκαν εξοφλημένοι λογαριασμοί για τα κριτήρια**
 1. Αν για τα συγκεκριμένα φίλτρα δεν βρεθούν Bills, το σύστημα εμφανίζει μήνυμα “Δεν βρέθηκαν εξοφλημένοι λογαριασμοί”.
 2. Ο χρήστης μπορεί να αλλάξει φίλτρα ή να επιστρέψει στο μενού.

Σχετικές FRs

- **FR-BL-02** - Διαχείριση κατάστασης Bill
- **FR-BL-03** - Πληρωμή Bill (ώστε να εμφανίζεται ως εξοφλημένος)
- **FR-ST-03** - Φιλτράρισμα statements ανά περίοδο (αν δίνετε δυνατότητα φίλτρων)

- **FR-BL-04** - Έλεγχος ληγμένου ή εξοφλημένου status
- **FR-UM-02** - Διαχείριση ρόλων (Company access)

STORYBOARDING

Παρακάτω πραγματοποιήθηκε ο σχεδιασμός των σεναρίων αλληλεπίδρασης του χρήστη με το υπο- ανάπτυξη e-banking σύστημα, σύμφωνα με το διάγραμμα Use Cases που παρουσιάστηκε παραπάνω. Παρακάτω θα παρουσιαστούν 3 βασικές λειτουργίες μέσω Paper Storyboard.

Τα storyboards αποτυπώνουν:

- την αρχική κατάσταση της οθόνης,
- τη δράση/επιλογή του χρήστη,
- την απόκριση του συστήματος,
- καθώς και τις διαδοχικές οθόνες που εμφανίζονται.

Στόχος των storyboards είναι να περιγράψουν με απλό τρόπο τη χρήση του συστήματος και ορίζεται ουσιαστικό βήμα πριν το GUI DESIGN & CODING.

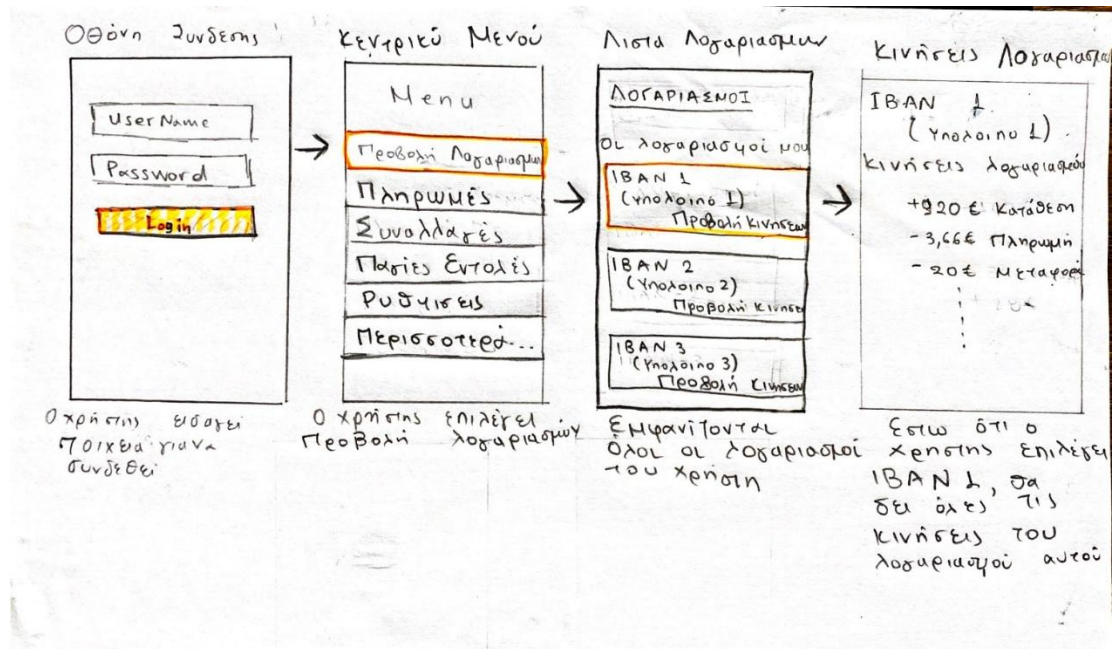
Στα επόμενα τμήματα παρουσιάζονται τα storyboards για τις επιλεγμένες βασικές λειτουργίες του συστήματος:

- Προβολή Λογαριασμών,
- Μεταφορά Χρημάτων,
- Ακύρωση Πάγιας Εντολής.

Κάθε storyboard περιλαμβάνει διαδοχικά frames με σαφή περιγραφή των ενεργειών του χρήστη και των αντιδράσεων του συστήματος.

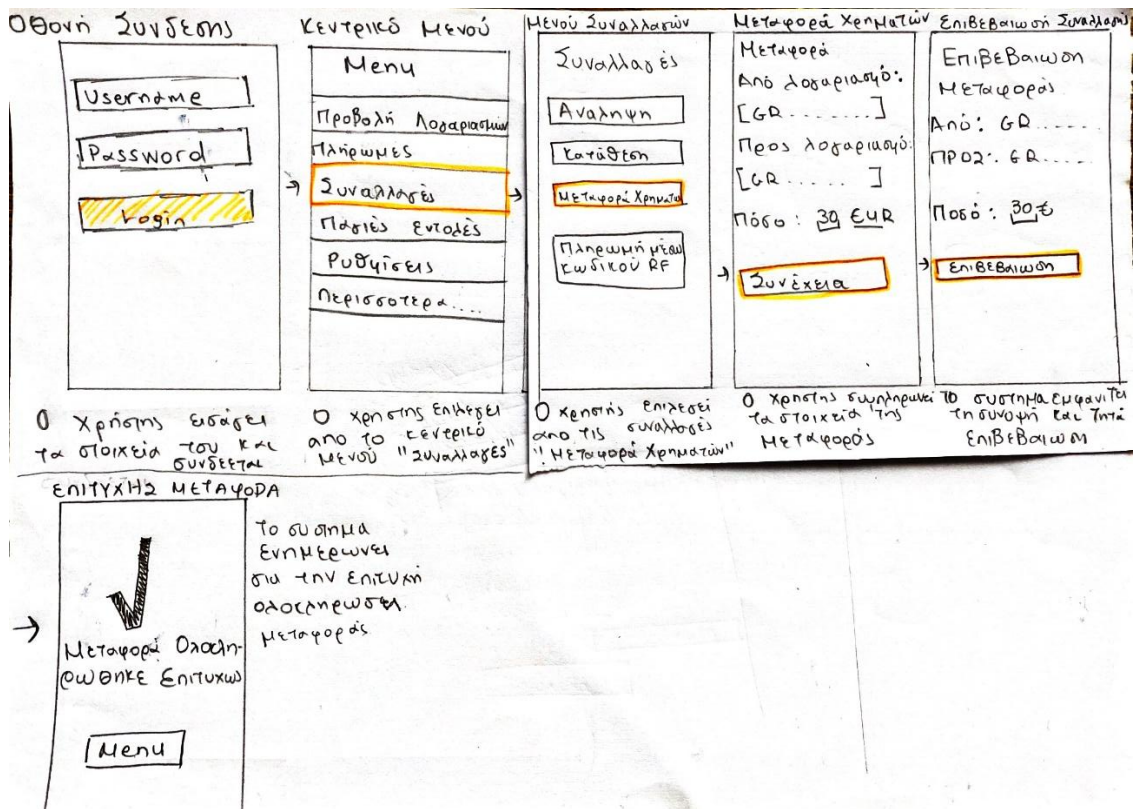
1^ο STORYBOARD

ΛΕΙΤΟΥΡΓΙΑ : ΠΡΟΒΟΛΗ ΛΟΓΑΡΙΑΣΜΩΝ



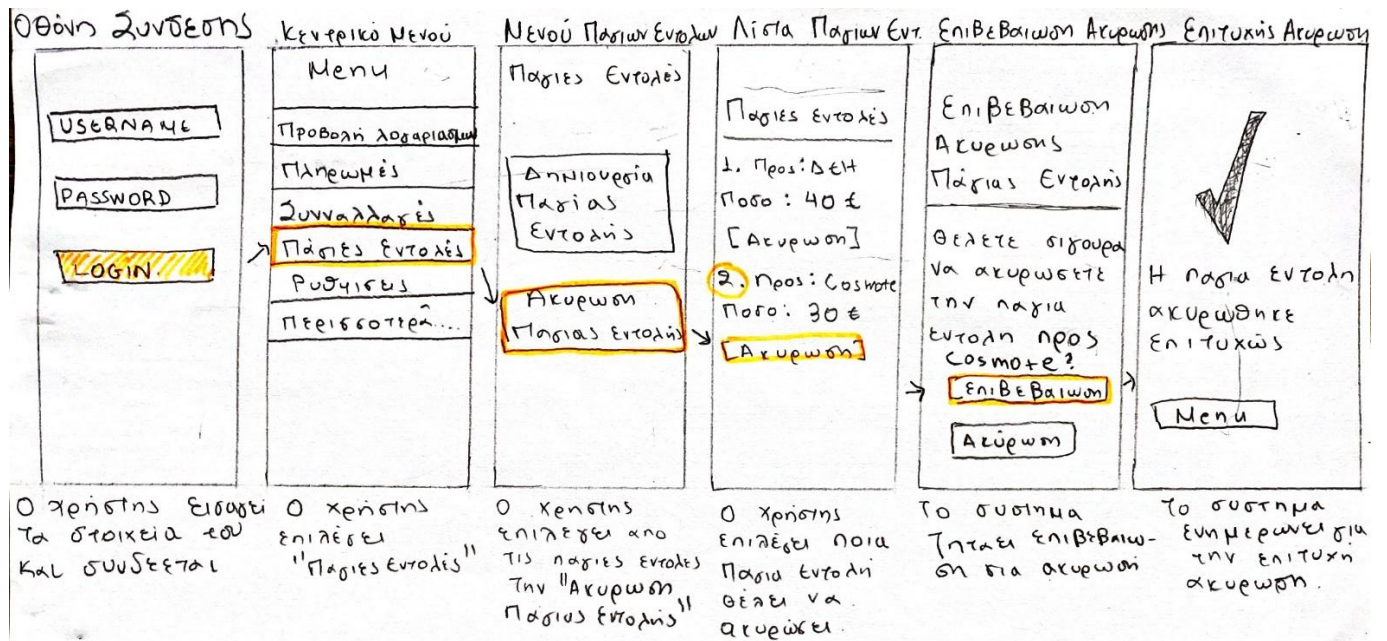
Το storyboard αυτό αποτυπώνει τη διαδικασία με την οποία ο χρήστης προβάλλει τους τραπεζικούς λογαριασμούς του μέσα από το e-banking σύστημα. Αφού εισέλθει με τα στοιχεία σύνδεσης, ο χρήστης μεταφέρεται στο κεντρικό μενού και επιλέγει τη λειτουργία «Προβολή Λογαριασμών». Το σύστημα εμφανίζει μια λίστα με όλους τους διαθέσιμους λογαριασμούς του, περιλαμβάνοντας πληροφορίες όπως IBAN και υπόλοιπο. Ο χρήστης μπορεί έπειτα να επιλέξει έναν συγκεκριμένο λογαριασμό για να δει αναλυτικά τις κινήσεις του.

Σενάριο GUI-01 - Προβολή Λογαριασμών και Κινήσεων
Καλύπτει τα Use Cases UC-01 και την προβολή κινήσεων λογαριασμού, και υλοποιεί τα FR: FR-UM-01, FR-UM-02, FR-AC-01, FR-ST-02.



Το storyboard αυτό περιγράφει τη ροή της λειτουργίας «Μεταφορά Χρημάτων», μέσω της οποίας ο χρήστης μπορεί να μεταφέρει χρήματα από έναν λογαριασμό του προς έναν άλλο λογαριασμό. Ο χρήστης εισέρχεται αρχικά στο σύστημα και από το κεντρικό μενού επιλέγει τη λειτουργία «Συναλλαγές» και στη συνέχεια «Μεταφορά Χρημάτων». Το σύστημα εμφανίζει την αντίστοιχη φόρμα, όπου ο χρήστης εισάγει τα στοιχεία της συναλλαγής, όπως τον λογαριασμό χρέωσης, τον λογαριασμό προορισμού και το ποσό. Ακολουθεί οθόνη επιβεβαίωσης των στοιχείων, ώστε ο χρήστης να βεβαιωθεί ότι όλα είναι σωστά πριν ολοκληρωθεί η συναλλαγή. Μετά την τελική αποδοχή, το σύστημα εκτελεί τη μεταφορά και εμφανίζει μήνυμα επιτυχούς ολοκλήρωσης.

Σενάριο GUI-02 - Εκτέλεση Μεταφοράς Χρημάτων
 Αντιστοιχεί στο Use Case UC-02 Μεταφορά Χρημάτων και υλοποιεί τα
 FR: FR-UM-01, FR-AC-01, FR-TX-03, FR-TX-05, FR-ST-01, FR-ST-02



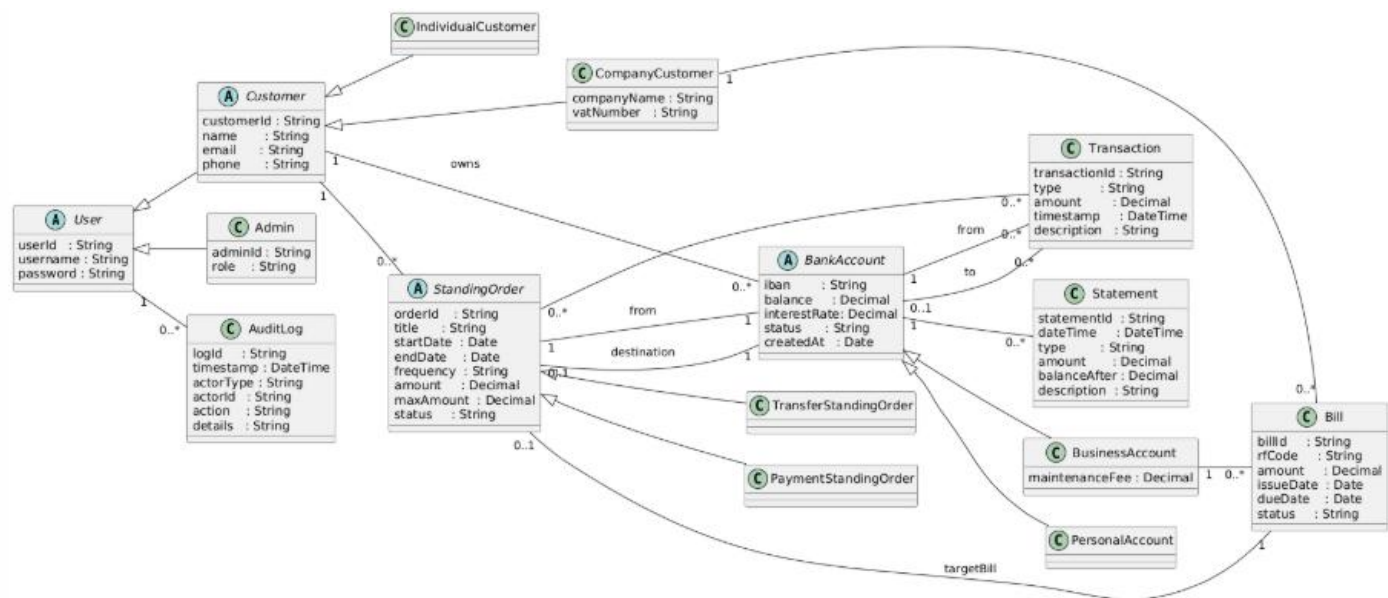
Το συγκεκριμένο storyboard παρουσιάζει τη διαδικασία ακύρωσης μιας πάγιας εντολής από τον χρήστη. Αρχικά ο χρήστης πραγματοποιεί είσοδο στο σύστημα και μεταφέρεται στο κεντρικό μενού. Από εκεί μεταφέρεται σε ένα υπο-μενού όπου διαλέγει τη λειτουργία «Ακύρωση Πάγιας Εντολής». Στη συνέχεια ο χρήστης επιλέγει ποια εντολή επιθυμεί να ακυρώσει και πατά το αντίστοιχο κουμπί «Ακύρωση». Το σύστημα εμφανίζει παράθυρο επιβεβαίωσης για να διασφαλίσει ότι η ακύρωση δεν έγινε καταλάθος. Μετά την τελική επιβεβαίωση, η εντολή διαγράφεται και το σύστημα ενημερώνει τον χρήστη ότι η ακύρωση ολοκληρώθηκε επιτυχώς.

Σενάριο GUI-03 - Ακύρωση Πάγιας Εντολής

Καλύπτει το Use Case "Ακύρωση Πάγιας Εντολής" και υλοποιεί τα FR:

FR-UM-01, FR-SO-05, FR-SO-06. Τα FR-SO-05 και FR-SO-06

προστέθηκαν συμπληρωματικά στον αρχικό κατάλογο απαιτήσεων, ώστε να καλύψουν πλήρως τη λειτουργικότητα της Διαχείρισης Πάγιας Εντολών.



Το παραπάνω διάγραμμα, παρουσιάζει το Domain Model , του ebanking της BANK OF TUC , αποτυπώνει λοιπόν τις κύριες επιχειρησιακές οντότητες , τα βασικά χαρακτηριστικά και πως σχετίζονται μεταξύ τους. Στο διάγραμμα περιλαμβάνεται οι εξής τύποι χρηστών (Individual Customer, Company Customer και Admin), τραπεζικοί λογαριασμοί (Personal και Business Accounts) καθώς και συναλλαγές, statements, bills για τους εταιρικούς πελάτες καθώς και τις πάγιες εντολές (Standing Orders) τόσο για μεταφορές όσο και για πληρωμές μέσω RF. Επιπλέον αποτυπώνεται ο μηχανισμός Audit Logging για την καταγραφή ενεργειών στο σύστημα. Οι πολλαπλότητες και οι σχέσεις μεταξύ των κλάσεων υποστηρίζουν τις λειτουργίες που περιγράφονται στα Use Cases και τις Λειτουργικές Απαιτήσεις (FRs), αποτυπώνοντας με συνέπεια τη δομή δεδομένων του συστήματος e-banking.

- **User:** Αντιπροσωπεύει κάθε χρήστη που συνδέεται στο σύστημα (πελάτης ή admin).
Πεδία: userId, username, password.
Σχέσεις :
 - Κάθε User (1) μπορεί να έχει 0..* εγγραφές AuditLog.
 - Κάθε AuditLog ανήκει ακριβώς σε 1 User.
- **Customer:** Γενική έννοια πελάτη της τράπεζας.
Πεδία: customerId, name, email, phone.
Σχέσεις :
 - Κληρονομεί από User (ιεραρχία, όχι πολλαπλότητες).
 - Ένας Customer (1) μπορεί να κατέχει 0..* BankAccount.
 - Ένας Customer (1) μπορεί να έχει 0..* StandingOrder.
 - Κάθε BankAccount και κάθε StandingOrder ανήκει σε ακριβώς 1 Customer.
- **IndividualCustomer:** Φυσικό πρόσωπο - πελάτης.
Πεδία: (όπως Customer).
Σχέσεις :
 - Υποκλάση του Customer (δεν αλλάζει τις πολλαπλότητες: 1 IndividualCustomer → 0..* λογαριασμοί, 0..* πάγιες εντολές).

- **CompanyCustomer:** Εταιρικός πελάτης (επιχείρηση).
Πεδία: companyName, vatNumber.
Σχέσεις :
 - Υποκλάση του Customer.
 - Ένας CompanyCustomer (1) μπορεί να εκδώσει 0..* Bill.
 - Κάθε Bill ανήκει σε ακριβώς 1 CompanyCustomer.

- **Admin:** Διαχειριστής του συστήματος.
Πεδία: adminId, role.
Σχέσεις :
 - Κληρονομεί από User.
 - Οι ενέργειές του καταγράφονται επίσης σε AuditLog (όπως κάθε User: 1 Admin → 0..* AuditLog entries).

- **AuditLog:** Καταγραφή ενεργειών στο σύστημα.
Πεδία: logId, timestamp, actorType, actorId, action, details.
Σχέσεις :
 - Κάθε AuditLog ανήκει σε ακριβώς 1 User (1).
 - Ένας User (1) μπορεί να έχει 0..* AuditLog.

- **BankAccount:** Τραπεζικός λογαριασμός.
Πεδία: iban, balance, interestRate, status, createdAt.
Σχέσεις :
 - Κάθε BankAccount ανήκει σε ακριβώς 1 Customer (1).
 - Ένας Customer (1) μπορεί να έχει 0..* BankAccount.
 - Ένας BankAccount (1) μπορεί να έχει 0..* Transaction ως “from” λογαριασμός.
 - Ένας BankAccount μπορεί να είναι “to” λογαριασμός για 0..* Transaction (η Transaction μπορεί να μην έχει “to”).
 - Ένας BankAccount (1) έχει 0..* Statement.
 - Ένας BankAccount (1) μπορεί να είναι λογαριασμός προέλευσης για 0..* StandingOrder (from).
 - Ένας BankAccount μπορεί να είναι λογαριασμός προορισμού για 0..* StandingOrder (destination), με πολλαπλότητα 0..1 στην πλευρά του λογαριασμού για κάθε StandingOrder.

- **PersonalAccount:** Προσωπικός λογαριασμός φυσικού προσώπου.
Πεδία: (ίδια με BankAccount).
Σχέσεις :
 - Υποκλάση του BankAccount, άρα ισχύει: 1 PersonalAccount → ανήκει σε 1 Customer (συνήθως IndividualCustomer), με 0..* Transactions/Statements.

- **BusinessAccount:** Εταιρικός λογαριασμός.
Πεδία: maintenanceFee.
Σχέσεις :
 - Υποκλάση του BankAccount.
 - Ένας BusinessAccount (1) μπορεί να σχετίζεται με 0..* Bill (π.χ. λογαριασμός στον οποίο πιστώνονται πληρωμές Bills).
 - Κάθε Bill συνδέεται με ακριβώς 1 BusinessAccount.

- **Transaction:** Μία συναλλαγή (κατάθεση, ανάληψη, μεταφορά, πληρωμή).
Πεδία: transactionId, type, amount, timestamp, description.
Σχέσεις :
 - Κάθε Transaction έχει έναν υποχρεωτικό λογαριασμό προέλευσης: from BankAccount (1).
 - Προαιρετικά έχει έναν λογαριασμό προορισμού: to BankAccount (0..1).
 - Ένας BankAccount (1) μπορεί να είναι from σε 0..* Transaction και to σε 0..* Transaction.
 - Μία Transaction μπορεί να συνδέεται με 0..* StandingOrder (π.χ. να έχει παραχθεί από πάγια εντολή).
 - Μία Transaction μπορεί να εμφανίζεται σε 1 ή περισσότερα Statement (ανάλογα πώς ορίζεται το statement στην υλοποίηση).

- **Statement:** Εγγραφή κίνησης λογαριασμού (γραμμή σε statement).
Πεδία: statementId, dateTime, type, amount, balanceAfter, description.
Σχέσεις :
 - Ένα Statement ανήκει σε ακριβώς 1 BankAccount (1).
 - Ένας BankAccount (1) έχει 0..* Statement.

- **StandingOrder:** Γενική οντότητα πάγιας εντολής.
Πεδία: orderId, title, startDate, endDate, frequency, amount, maxAmount, status.
Σχέσεις :
 - Κάθε StandingOrder ανήκει σε 1 Customer (1). Ένας Customer (1) μπορεί να έχει 0..* StandingOrder.
 - Κάθε StandingOrder έχει έναν λογαριασμό προέλευσης: from BankAccount (1).
 - Μπορεί να έχει προαιρετικά έναν λογαριασμό προορισμού: destination BankAccount (0..1).
 - Μπορεί προαιρετικά να στοχεύει ένα Bill: targetBill (0..1 Bill).
 - Μία StandingOrder μπορεί να δημιουργήσει 0..* Transaction (κάθε εκτέλεση της εντολής).

- **TransferStandingOrder:** Πάγια εντολή μεταφοράς χρημάτων μεταξύ λογαριασμών.
Πεδία: (ίδια με StandingOrder).
Σχέσεις :
 - Υποκλάση του StandingOrder. Συνήθως έχει υποχρεωτικό destination BankAccount (1) για να γίνεται η μεταφορά.

- **PaymentStandingOrder:** Πάγια εντολή πληρωμής Bills.
Πεδία: (ίδια με StandingOrder).
Σχέσεις :
 - Υποκλάση του StandingOrder.
 - Συνδέεται με 1 Bill ως targetBill (κάθε PaymentStandingOrder → 0..1 Bill, κάθε Bill μπορεί να έχει 0..* συνδεδεμένες PaymentStandingOrder).

- **Bill:** Λογαριασμός πληρωμής (ηλεκτρικό, τηλέφωνο κ.λπ.).
Πεδία: billId, rfCode, amount, issueDate, dueDate, status.
Σχέσεις :
 - Κάθε Bill ανήκει σε 1 CompanyCustomer (εκδότης). Ένας CompanyCustomer (1) μπορεί να έχει 0..* Bill.

- Κάθε Bill πιστώνεται σε 1 BusinessAccount (1). Ένας BusinessAccount (1) μπορεί να σχετίζεται με 0..* Bill.
- Ένα Bill μπορεί να είναι στόχος 0..* PaymentStandingOrder (ή καμίας).

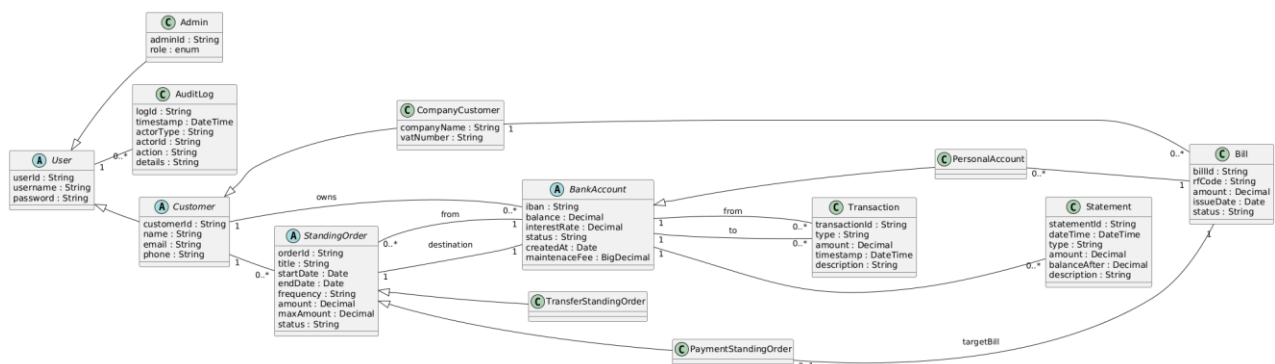
3^η (Τελική Φάση) :

- Στην 3η φάση ζητήθηκε ο λεπτομερής σχεδιασμός και η πλήρης υλοποίηση του συστήματος e-banking ως desktop εφαρμογή σε Java. Συγκεκριμένα, έπρεπε να επεκταθεί το διάγραμμα κλάσεων (προσθήκη business logic και UI), να εφαρμοστούν τα πρότυπα Singleton, Factory, Command, Builder, Bridge και DAO, και να υλοποιηθεί αποθήκευση δεδομένων μέσω αρχείων CSV/JSON. Ταυχρόνα διορθώσαμε τυχόν παραλείψεις και λάθη που εντοπίστηκαν στην προηγούμενη φάση προσαρμόζοντας έτσι το τελικό μας project.

3.1 Αλλαγές/προσθήκες που πραγματοποιήθηκαν :

- Αλλαγή του τύπου του Role(από String σε enum)
- Διαγραφή του Individual Customer (με την διαμόρφωση του UML diagram που είχαμε κάνει δεν υπήρχε κάποιος λόγος για την ύπαρξη αντίστοιχης κλάσης.)
- Αλλαγή των amount και maxAccount από big decimal->integer
- Δημιουργία numeration (enum) για τα πιθανά actions του κάθε χρήστη στα StandingOrderType.
- Προσθήκη ενός πάνελ με όρους και προϋποθέσεις .
- Αφαίρεση των κουμπιών ανάληψης/κατάθεσης καθώς δεν χρειάζονται σε μία ηλεκτρονική τραπεζική εφαρμογή.
- Διαγραφή BusinessAccount

- Ανανεωμένο uml diagram



3.2 Επεξήγηση της πρακτικής υλοποίησης (κώδικας) :

Η πρακτική υλοποίηση του συστήματος e-banking πραγματοποιήθηκε σε **Java** ως desktop εφαρμογή. Ο κώδικας οργανώθηκε με διαχωρισμό ανά επίπεδο (μοντέλο δεδομένων, επιχειρησιακή λογική και πρόσβαση/αποθήκευση δεδομένων), ώστε να είναι πιο εύκολη η συντήρηση και η μελλοντική επέκταση του συστήματος.

Ο κώδικας χωρίστηκε σε ενότητες (packages) με σαφή ρόλο: το **μοντέλο/οντότητες (entities/model)** περιλαμβάνει τις βασικές κλάσεις του συστήματος (χρήστες, λογαριασμοί, συναλλαγές, statements, κ.λπ.), η **επιχειρησιακή λογική (services)** υλοποιεί τις λειτουργίες του e-banking (login/ρόλοι, προβολή λογαριασμών, συναλλαγές, ενημερώσεις υπολοίπων και καταγραφή κινήσεων), ενώ το επίπεδο **πρόσβασης δεδομένων (DAO / persistence)** αναλαμβάνει τη φόρτωση και αποθήκευση πληροφοριών χωρίς να εκθέτει τις λεπτομέρειες αποθήκευσης στα services. Επιπλέον χρησιμοποιήθηκαν βοηθητικές κλάσεις/ενότητες (factories/utilities) για κεντρική δημιουργία και διαχείριση αντικειμένων.

Για την αποθήκευση των δεδομένων χρησιμοποιούνται αρχεία **CSV**. Τα αρχεία δεδομένων τοποθετούνται σε συγκεκριμένο φάκελο του project (π.χ. **resources**) και προσπελαύνονται μέσω **relative file paths**, ώστε η εφαρμογή να εκτελείται σωστά και σε διαφορετικό υπολογιστή χωρίς αλλαγές. Οι αλλαγές που προκύπτουν από ενέργειες του χρήστη (π.χ. νέα συναλλαγή, ενημέρωση υπολοίπων, αλλαγή κατάστασης) αποθηκεύονται ξανά στα αρχεία, διατηρώντας συνέπεια μεταξύ εκτέλεσης και δεδομένων.

Σε επίπεδο λειτουργικότητας, υλοποιήθηκαν οι βασικές ροές του συστήματος: **ταυτοποίηση χρήστη (login)** και **διάκριση ρόλων, προβολή λογαριασμών, εκτέλεση συναλλαγών** (μεταφορά/πληρωμή), καθώς και η **δημιουργία και ενημέρωση κινήσεων (statements)** μετά από κάθε συναλλαγή. Όπου απαιτείται εφαρμόζονται έλεγχοι εγκυρότητας σε σημεία όπως το log in και στη συνέχεια πραγματοποιούνται οι αντίστοιχες ενημερώσεις στα δεδομένα.

Και τέλος, πραγματοποιήθηκε υλοποίηση Design Patterns και στο frontend.

3.3 Παραλείψεις που εντοπίστηκαν:

Κάποια από τις παραλείψεις που εντοπίστηκαν και θα βοηθούσαν στην καλύτερη και πιο ακριβής υλοποίηση ενός αντίστοιχου project (Bank of TUC) θα ήταν:

- Η δημιουργία ενός registry για εγγραφές νέων πελατών
- Υλοποίηση design patterns
- Διαγραφή πάγιας εντολής
- Έλεγχοι εγκυρότητας και σύνδεση τους παράλληλα και με frontend και backend στο πρακτικό κομμάτι υλοποίησης.